B. K. Birla College, Kalyan
(Empowered Autonomous Status)
Department of Information Technology
Subject: Introduction to Artificial Intelligence

Index

Sr. No.	Particular	Date	Page No.	Sign
1.	To implement Breadth-First Search (BFS) traversal on a graph using Python and to display the order in which nodes are visited.	02nd September 2026	**2**	
2.	To implement Depth-First Search (DFS) traversal on a graph using Python and to display the order in which nodes are visited.	05th September 2026	**4**	
3.	To solve the Water Jug Problem using Breadth First Search (BFS).	08th September 2026	**7**	
4.	To implement A* Search traversal on a graph using Python and to display the order in which nodes are visited.	10th September 2026	**9**	
5.	To implement Greedy Best First Search (GBFS) traversal on a graph using Python and to display the order in which nodes are visited.	19th September 2026	**11**	
6.	To implement the Minimax search algorithm for 2-player games. You may use a game tree with 2 plies.	22nd September 2026	**13**	
7.	To implement a basic rule-based expert system that classifies weather conditions using temperature, humidity, and wind speed through IF–THEN production rules.	25th September 2026	**15**	
8.	To write a case study on Waymo's Self-Driving Car AI System.	28th September 2026	**19**	

Practical No. 01

Aim:

To implement Breadth-First Search (BFS) traversal on a graph using Python and to display the order in which nodes are visited.

Background Information:

Breadth-First Search (BFS) is a graph traversal algorithm that starts at a source node and explores all its neighbours before moving to the next level of neighbours — traversing the graph "layer by layer."

Key Points:

- Uses a Queue (FIFO) to decide which node to visit next
- Uses a visited set to avoid revisiting nodes and infinite loops
- Explores nodes in increasing order of distance from the source, so it finds the shortest path in unweighted graphs
- Contrasts with DFS, which goes deep into one branch before backtracking

Applications: shortest path finding, web crawlers, social network connections, network broadcasting, puzzle solving.

Complexity: Time — O(V + E), Space — O(V), where V = vertices, E = edges.

Code:

```
from collections import deque

class Node:
    def __init__(self, value):
        self.value = value
        self.children = []

# Build the tree from your diagram
A = Node('A')
B = Node('B')
C = Node('C')
D = Node('D')
E = Node('E')
F = Node('F')
G = Node('G')

A.children = [B, C]
B.children = [D, E]
C.children = [F, G]

def bfs(root):
    visit_order = []
    queue = deque([root])       # FIFO queue, start with root
```

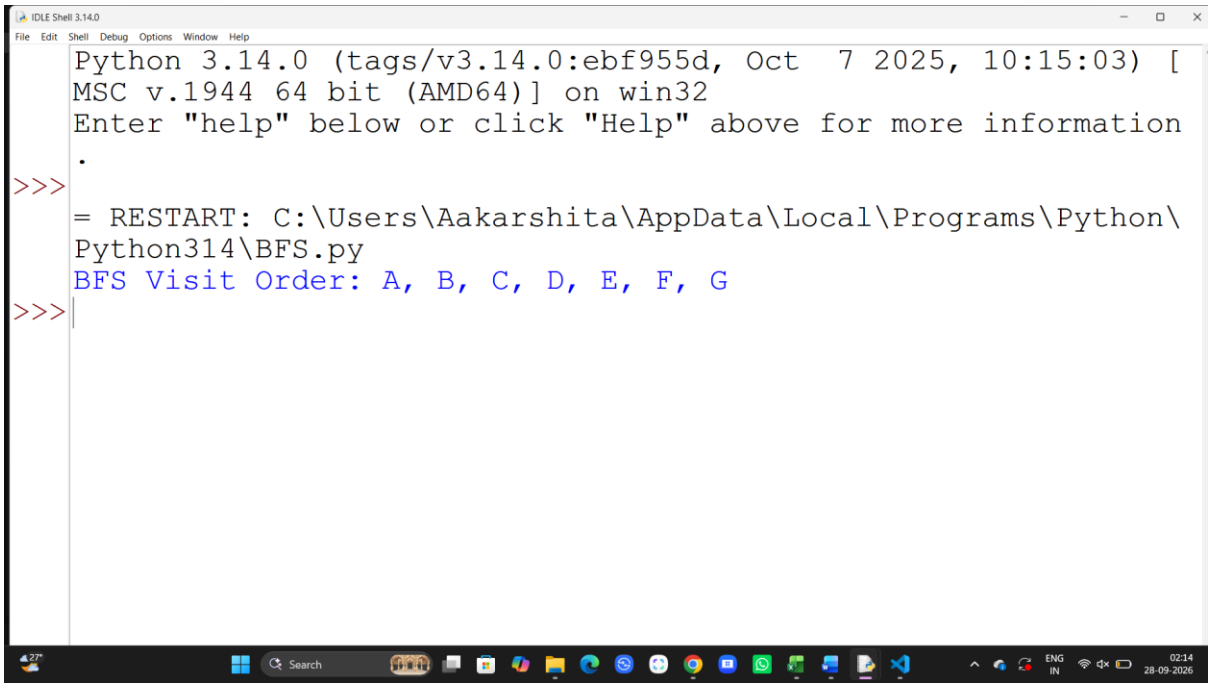
```
    while queue:
        node = queue.popleft()  # remove from front
        visit_order.append(node.value)

        for child in node.children:
            queue.append(child) # add children to back

    return visit_order

print("BFS Visit Order:", ", ".join(bfs(A)))
```

Output:



Practical No. 02

Aim:

To implement Depth-First Search (DFS) traversal on a graph using Python and to display the order in which nodes are visited.

Background Information:

Breadth-First Search (BFS) is a graph traversal algorithm that starts at a source node and explores all its neighbours before moving to the next level of neighbours — traversing the graph "layer by layer."

Key Points:

- Uses a Queue (FIFO) to decide which node to visit next
- Uses a visited set to avoid revisiting nodes and infinite loops
- Explores nodes in increasing order of distance from the source, so it finds the shortest path in unweighted graphs
- Contrasts with DFS, which goes deep into one branch before backtracking

Applications: shortest path finding, web crawlers, social network connections, network broadcasting, puzzle solving.

Complexity: Time — O(V + E), Space — O(V), where V = vertices, E = edges.

Code:

For Recursive DFS:

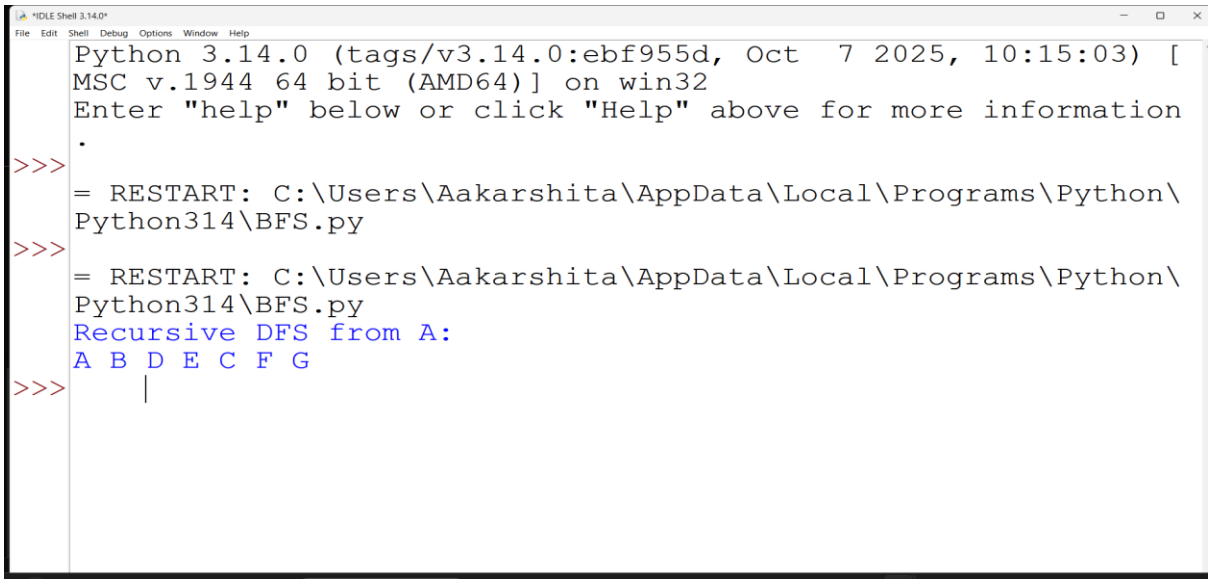
```
# Graph represented as an adjacency list (tree from the sketch)
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F', 'G'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C'],
    'G': ['C']
}
# ---------- Recursive DFS ----------
def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node, end=' ')
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)
    return visited
print("Recursive DFS from A:")
dfs_recursive(graph, 'A')
```

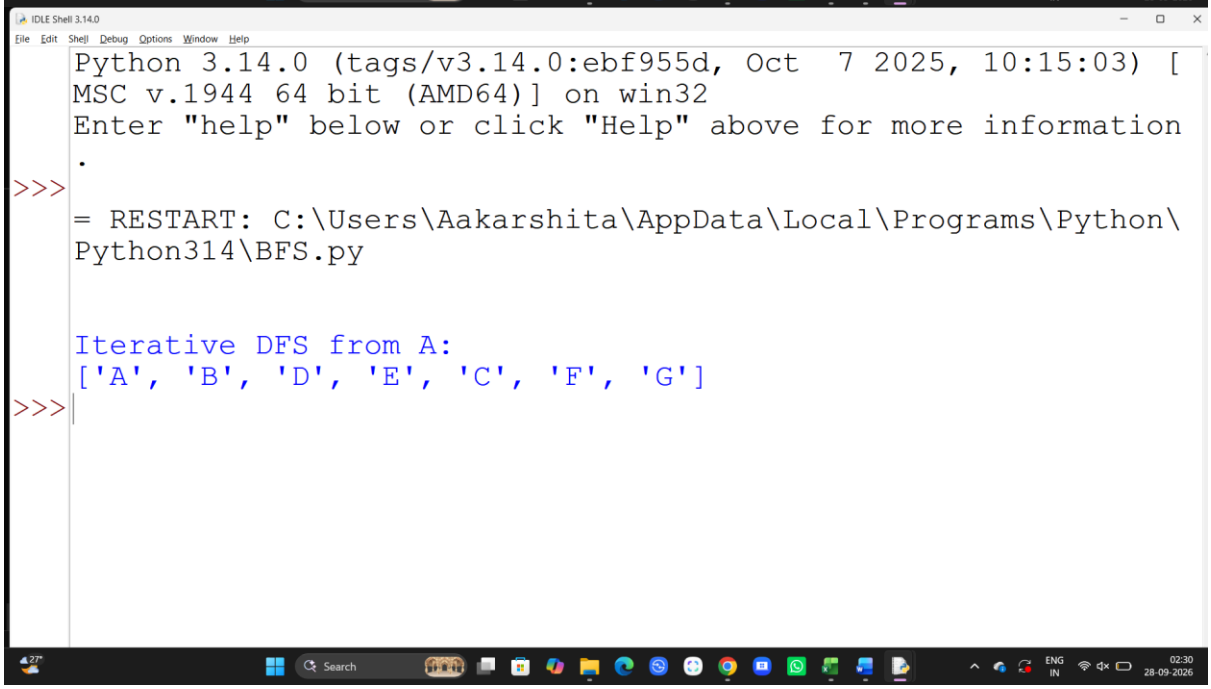
For Iterative DFS:

```
# Graph represented as an adjacency list (tree from the sketch)
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F', 'G'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C'],
    'G': ['C']
}

# ---------- Iterative DFS (using a stack) ----------
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    order = []
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            order.append(node)
            for neighbor in reversed(graph[node]):
                if neighbor not in visited:
                    stack.append(neighbor)
    return order
print("\n\nIterative DFS from A:")
print(dfs_iterative(graph, 'A'))
```

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [
MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information
.
>>>
= RESTART: C:\Users\Aakarshita\AppData\Local\Programs\Python\
Python314\BFS.py
>>>
= RESTART: C:\Users\Aakarshita\AppData\Local\Programs\Python\
Python314\BFS.py
Recursive DFS from A:
A B D E C F G
>>>
Search
ENG
IN
02:28
28-09-2026


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [
MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information
.
>>>
= RESTART: C:\Users\Aakarshita\AppData\Local\Programs\Python\
Python314\BFS.py
Iterative DFS from A:
['A', 'B', 'D', 'E', 'C', 'F', 'G']
>>>
Search
ENG
IN
02:30
28-09-2026

Practical No. 03

Aim:

To solve the Water Jug Problem using Breadth First Search (BFS).

Background Information:

The Water Jug Problem is a classic AI search problem where you're given two (or more) jugs of different fixed capacities (with no measuring marks) and an unlimited water source. The goal is to measure out a specific target quantity of water using a sequence of allowed operations.

Key Points:

- Each **state** is represented as a pair (x, y) — the current amount of water in Jug1 and Jug2
- **Allowed operations (moves):**
    1. Fill a jug completely
    2. Empty a jug completely
    3. Pour water from one jug into another (until the source is empty or the destination is full)
- BFS is used because it explores all possible states level by level, guaranteeing the **shortest sequence of operations** (minimum number of steps) to reach the target/goal state
- A **visited set** is maintained to avoid revisiting the same jug-state and to prevent infinite loops
- The **goal state** is reached when either jug contains the required target amount of water

Why BFS is suitable here:

Since every operation has equal "cost" (each move counts as one step), the problem is essentially an unweighted state-space search — making BFS ideal for finding the optimal (shortest) solution path, same way it does in graph traversal.

Code:

```
from collections import deque
from math import gcd

def water_jug_bfs(cap1, cap2, target):
    # Check solvability first
    if target > max(cap1, cap2) or target % gcd(cap1, cap2) != 0:
        return None
    start = (0, 0)
    visited = {start}
    queue = deque([(start, [start])])  # (state, path)

    while queue:
        (j1, j2), path = queue.popleft()
```

```
        if j1 == target or j2 == target:
            return path

        possible_states = [
            (cap1, j2),                                     # Fill Jug1
            (j1, cap2),                                     # Fill Jug2
            (0, j2),                                       # Empty Jug1
            (j1, 0),                                       # Empty Jug2
            (j1 - min(j1, cap2 - j2), j2 + min(j1, cap2 - j2)),          # Pour Jug1 -> Jug2
            (j1 + min(j2, cap1 - j1), j2 - min(j2, cap1 - j1)),          # Pour Jug2 -> Jug1
        ]

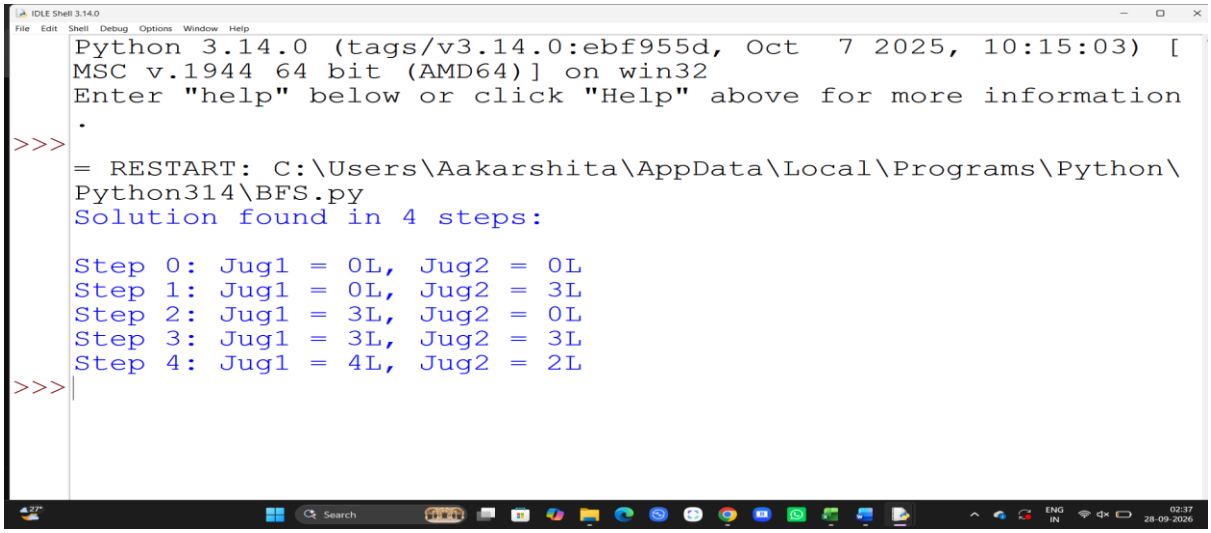
        for state in possible_states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))
    return None

if __name__ == "__main__":
    cap1, cap2, target = 4, 3, 2

    solution = water_jug_bfs(cap1, cap2, target)

    if solution:
        print(f"Solution found in {len(solution) - 1} steps:\n")
        for i, state in enumerate(solution):
            print(f"Step {i}: Jug1 = {state[0]}L, Jug2 = {state[1]}L")
    else:
        print("No solution exists for these jug capacities and target.")
```

Output:



Practical No. 04

Aim:

To implement A* Search traversal on a graph using Python and to display the order in which nodes are visited.

Background Information:

A* (A-star) is an informed (heuristic-based) search algorithm used to find the shortest path between a start node and a goal node in a weighted graph. It combines the strengths of Dijkstra's Algorithm (guaranteed shortest path) and Greedy Best-First Search (fast, goal-directed search) using a heuristic function.

Key Points:

- Uses an evaluation function: **f(n) = g(n) + h(n)**
  - **g(n)** = actual cost from the start node to node *n*
  - **h(n)** = heuristic estimate of cost from *n* to the goal
  - **f(n)** = estimated total cost of the cheapest path through *n*
- Uses a **priority queue (min-heap)** to always expand the node with the lowest f(n) value next
- A **visited/closed set** keeps track of already-evaluated nodes
- The heuristic must be **admissible** (never overestimates the true cost) to guarantee an optimal solution
- Contrasts with BFS/DFS, which don't consider edge weights or use any goal-directed estimation

Applications: pathfinding in games and robotics, GPS/map navigation systems, puzzle solving (e.g., 8-puzzle, 15-puzzle), network routing.

Complexity: Time and Space — O(b^d) in the worst case (b = branching factor, d = depth of solution), but typically much better in practice due to the heuristic guiding the search.

Code:

```
import heapq

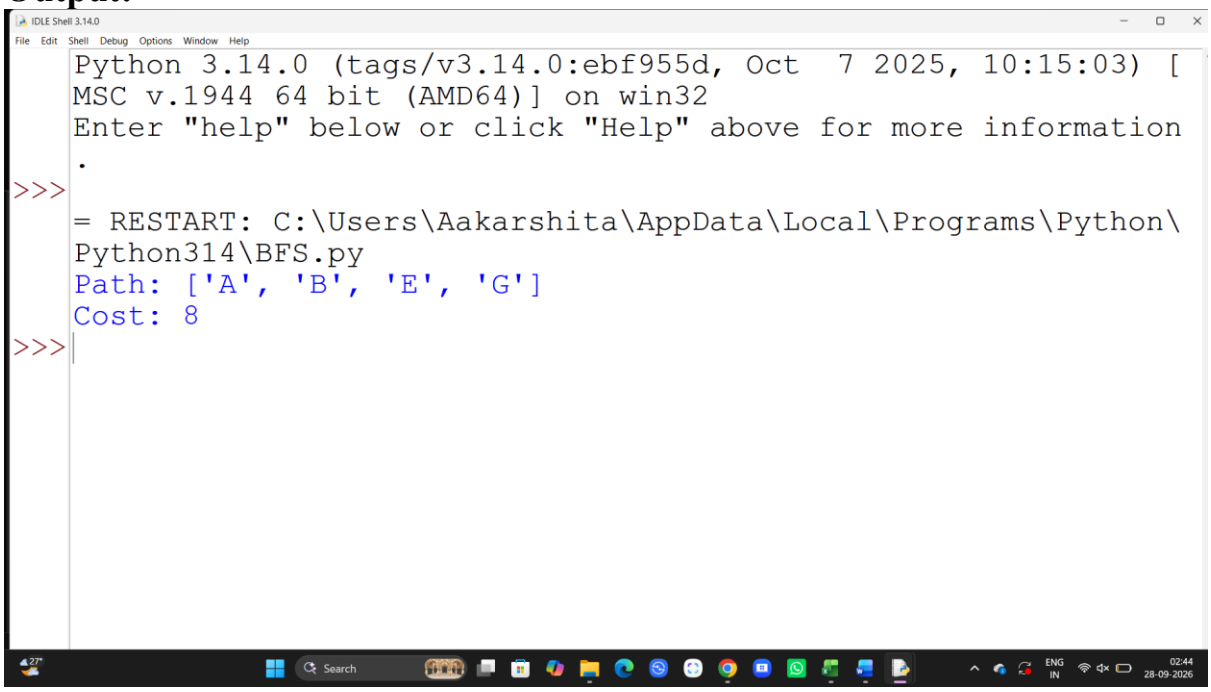

def astar(graph, start, goal, h):
    open_set = [(h[start], start, [start], 0)]
    while open_set:
        f, node, path, g = heapq.heappop(open_set)
        if node == goal:
            return path, g
        for neighbor, cost in graph[node]:
            new_g = g + cost
            heapq.heappush(open_set, (new_g + h[neighbor], neighbor, path + [neighbor],
new_g))
    return None, float('inf')
```

```
graph = {
   'A': [('B', 1), ('C', 4)],
   'B': [('D', 2), ('E', 5)],
   'C': [('F', 3)],
   'D': [('G', 6)],
   'E': [('G', 2)],
   'F': [('G', 1)],
   'G': []
}

h = {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0, 'F': 0, 'G': 0}  # no heuristic info

path, cost = astar(graph, 'A', 'G', h)
print("Path:", path)
print("Cost:", cost)
```

Output:



Practical No. 05

Aim:

To implement Greedy Best First Search (GBFS) traversal on a graph using Python and to display the order in which nodes are visited.

Background Information:

Greedy Best-First Search (GBFS) is an informed search algorithm that expands the node which appears **closest to the goal**, based purely on a heuristic function — without considering the cost already spent to reach that node.

Key Points:

- Uses only the heuristic function: **f(n) = h(n)**
  - **h(n)** = estimated cost from node *n* to the goal (no g(n) component, unlike A*)
- Uses a **priority queue (min-heap)** to always expand the node with the lowest h(n) value next
- A **visited set** avoids revisiting nodes and infinite loops
- Faster than A* in many cases since it ignores path cost, but **does not guarantee the shortest/optimal path**
- Can get stuck in poor paths since it's "greedy" and only looks at what seems best locally

Applications: pathfinding where speed matters more than optimality, game AI, robotics navigation, puzzle solving where near-optimal solutions are acceptable.

Complexity: Time and Space — O(b^m) in the worst case (b = branching factor, m = maximum depth), but generally faster in practice than uninformed search due to heuristic guidance.

Code:

```
import heapq
def greedy_best_first_search(graph, start, goal, heuristic):
    # Priority queue stores (heuristic_value, node)
    open_list = [(heuristic[start], start)]
    came_from = {start: None}
    visited = set()

    while open_list:
        _, current = heapq.heappop(open_list)
        if current == goal:
            return reconstruct_path(came_from, current)
        if current in visited:
            continue
        visited.add(current)

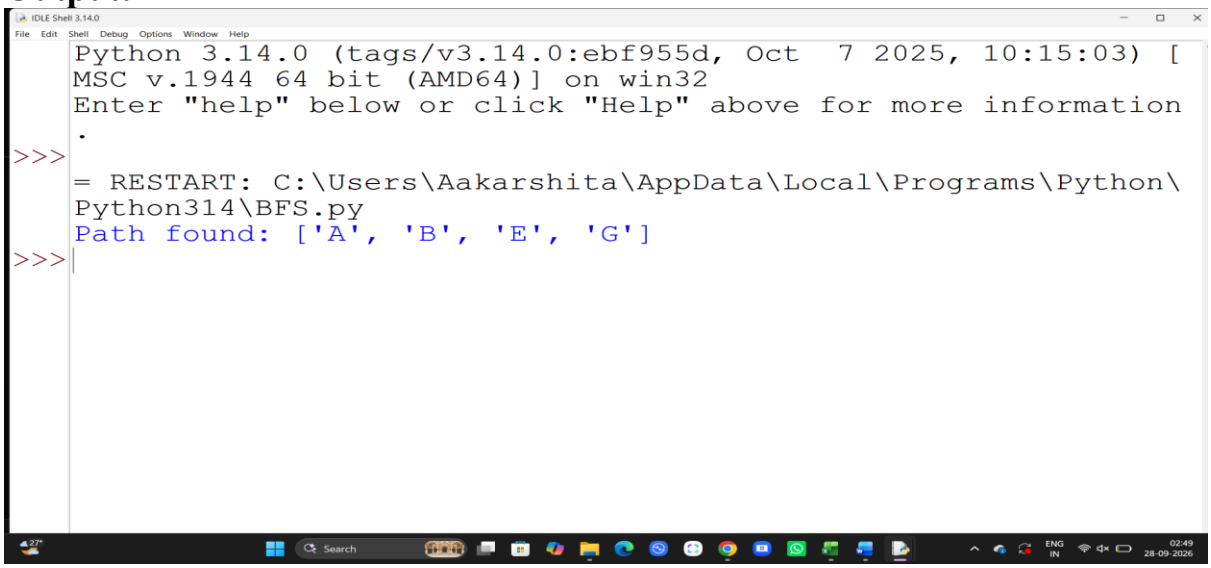
        for neighbor in graph.get(current, []):
```

```
            if neighbor not in visited:
                came_from.setdefault(neighbor, current)
                heapq.heappush(open_list, (heuristic[neighbor], neighbor))
    return None  # No path found

def reconstruct_path(came_from, current):
    path = [current]
    while came_from[current] is not None:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path
# Example usage
if __name__ == "__main__":
    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': ['G'],
        'F': ['G'],
        'G': []
    }
    # Estimated distance to goal 'G' (e.g., straight-line distance)
    heuristic = {
        'A': 6, 'B': 4, 'C': 4,
        'D': 5, 'E': 2, 'F': 1, 'G': 0
    }

    path = greedy_best_first_search(graph, 'A', 'G', heuristic)
    print("Path found:", path)
```

Output:



Practical No. 06

Aim:

To implement the Minimax search algorithm for 2-player games. You may use a game tree with 2 plies.

Background Information:

Minimax is a decision-making algorithm used in two-player, zero-sum games (like Tic-Tac-Toe, Chess, Checkers) to determine the optimal move for a player, assuming the opponent also plays optimally.

Key Points:

- Involves two players: **Maximizer** (tries to maximize the score) and **Minimizer** (tries to minimize the score)
- Builds a **game tree** where each node represents a game state, and explores it recursively (depth-first) down to terminal states (win/loss/draw) or a set depth limit
- At each level, the algorithm alternates between maximizing and minimizing the value, **backing up** the best value from child nodes to the parent
- Assumes the opponent always plays their best possible move — hence the name "mini-max" (minimize the maximum possible loss)
- Often combined with **Alpha-Beta Pruning** to eliminate branches that won't affect the final decision, improving efficiency

Applications: two-player board games (Chess, Tic-Tac-Toe, Checkers, Go), game AI decision-making, adversarial search problems.

Complexity: Time — O(b^d), Space — O(b·d) (b = branching factor, d = depth of tree); with Alpha-Beta Pruning, time complexity can improve to O(b^(d/2)) in the best case.

Code:

```
class Node:
    def __init__(self, name, children=None, value=None):
        self.name = name
        self.children = children or []
        self.value = value
        self.is_leaf = len(self.children) == 0

def minimax(node, is_maximizing):
    # Base case: leaf node -> just return its raw value
    if node.is_leaf:
        return node.value

    if is_maximizing:
        best_value = float('-inf')
        for child in node.children:
            value = minimax(child, False)   # next turn is MIN
```

```
            best_value = max(best_value, value)
        return best_value
    else:
        best_value = float('inf')
        for child in node.children:
            value = minimax(child, True)    # next turn is MAX
            best_value = min(best_value, value)
        return best_value

def best_move(root):
    best_val = float('-inf')
    best_child = None
    for child in root.children:
        val = minimax(child, False)   # after MAX's first move, it's MIN's turn
        if val > best_val:
            best_val = val
            best_child = child
    return best_child, best_val

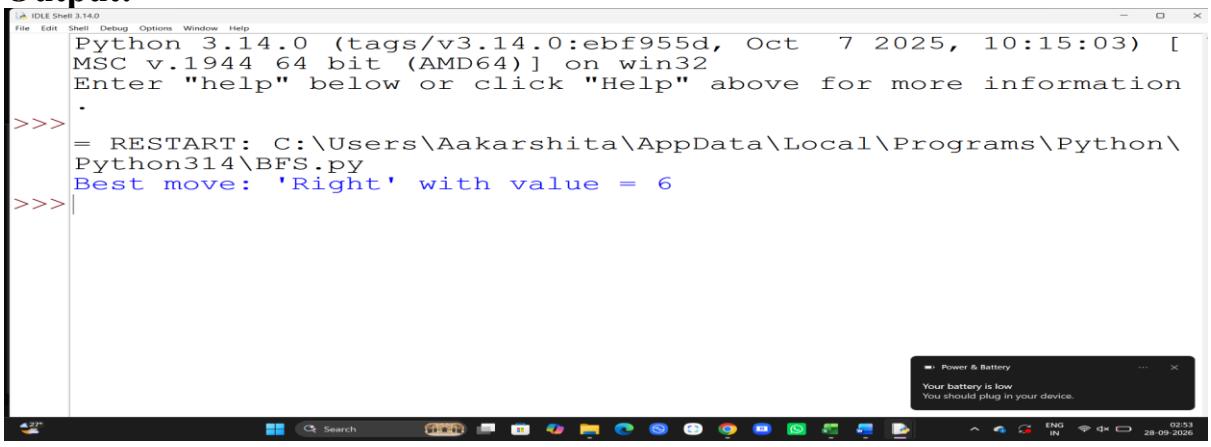
# Leaf nodes
leaf_3 = Node("Left-Left",  value=3)
leaf_5 = Node("Left-Right", value=5)
leaf_6 = Node("Right-Left", value=6)
leaf_9 = Node("Right-Right", value=9)

# OPPONENT (MIN) — ply 2
Left  = Node("Left",  children=[leaf_3, leaf_5])
Right = Node("Right", children=[leaf_6, leaf_9])

# YOU (MAX) — root, ply 1
root = Node("ROOT", children=[Left, Right])

move, value = best_move(root)
print(f"Best move: '{move.name}' with value = {value}")
```

Output:



Practical No. 07

Aim:

To implement a basic rule-based expert system that classifies weather conditions using temperature, humidity, and wind speed through IF–THEN production rules.

Background Information:

A Rule-Based Expert System is an AI system that mimics human decision-making by applying a set of **IF–THEN production rules** to a given set of facts (inputs) in order to draw conclusions or make classifications, without needing complex learning algorithms.

Key Points:

- **Knowledge Base**: Stores domain facts and IF–THEN rules (e.g., "IF temperature > 35°C THEN weather = Hot")
- **Inference Engine**: Applies the rules to the given inputs to derive conclusions — commonly using **forward chaining** (data-driven: start from known facts and apply rules to reach a conclusion)
- **Facts/Input Parameters**: In this case — temperature, humidity, and wind speed — act as the working memory the rules are evaluated against
- **Production Rules**: Simple conditional statements of the form IF <condition> THEN <action/conclusion>, chained together to classify the overall weather condition
- **Explanation Facility** (optional): Can justify why a particular conclusion was reached, based on which rules fired

How Classification Works Here:

The system checks temperature, humidity, and wind speed against predefined threshold rules (e.g., high temp + low humidity → "Hot and Dry", low temp + high wind → "Cold and Windy") and fires the matching rule(s) to output a weather classification.

Applications: medical diagnosis systems, fault diagnosis, financial advisory systems, agricultural advisory systems, basic weather advisories, troubleshooting systems.

Complexity: Time — O(n) to O(n^2) depending on rule matching strategy (n = number of rules); Space — O(n) for storing rules and facts.

Code:

Python Program – Basic Version

```
# Basic Rule-Based Expert System
# Weather Classification

def weather_expert_system(temperature, humidity, wind_speed):

    # Rule 1: Stormy weather
    if wind_speed >= 50:
        return "Stormy Weather"
```

```
    # Rule 2: Rainy weather
    elif humidity >= 80 and 20 <= temperature <= 35:
        return "Rainy Weather"

    # Rule 3: Hot weather
    elif temperature >= 35:
        return "Hot Weather"

    # Rule 4: Cold weather
    elif temperature < 20:
        return "Cold Weather"

    # Rule 5: Pleasant weather
    elif 25 <= temperature < 35 and humidity < 60:
        return "Pleasant Weather"

    # Rule 6: Cloudy weather
    elif 20 <= temperature < 35 and 60 <= humidity < 80:
        return "Cloudy Weather"

    # Default rule
    else:
        return "Moderate Weather"


print("====================================")
print("       WEATHER EXPERT SYSTEM")
print("====================================")

temperature = float(input("Enter temperature (°C): "))
humidity = float(input("Enter humidity (%): "))
wind_speed = float(input("Enter wind speed (km/h): "))

result = weather_expert_system(
    temperature, humidity, wind_speed
)

print("\nWeather Classification:", result)
```

Python Program – Version Showing the Inference Process

```
def weather_expert_system(temp, humidity, wind):

    print("\n--- Inference Process ---")

    if wind >= 50:
        print("Rule R1 fired: IF wind speed >= 50 THEN Stormy")
        return "Stormy Weather"
```

```
    elif humidity >= 80 and 20 <= temp <= 35:
        print("Rule R2 fired: IF humidity >= 80 AND temperature is 20-35 THEN Rainy")
        return "Rainy Weather"

    elif temp >= 35:
        print("Rule R3 fired: IF temperature >= 35 THEN Hot")
        return "Hot Weather"

    elif temp < 20:
        print("Rule R4 fired: IF temperature < 20 THEN Cold")
        return "Cold Weather"

    elif 25 <= temp < 35 and humidity < 60:
        print("Rule R5 fired: IF temperature is 25-35 AND humidity < 60 THEN Pleasant")
        return "Pleasant Weather"

    elif 20 <= temp < 35 and 60 <= humidity < 80:
        print("Rule R6 fired: IF temperature is 20-35 AND humidity is 60-79 THEN Cloudy")
        return "Cloudy Weather"

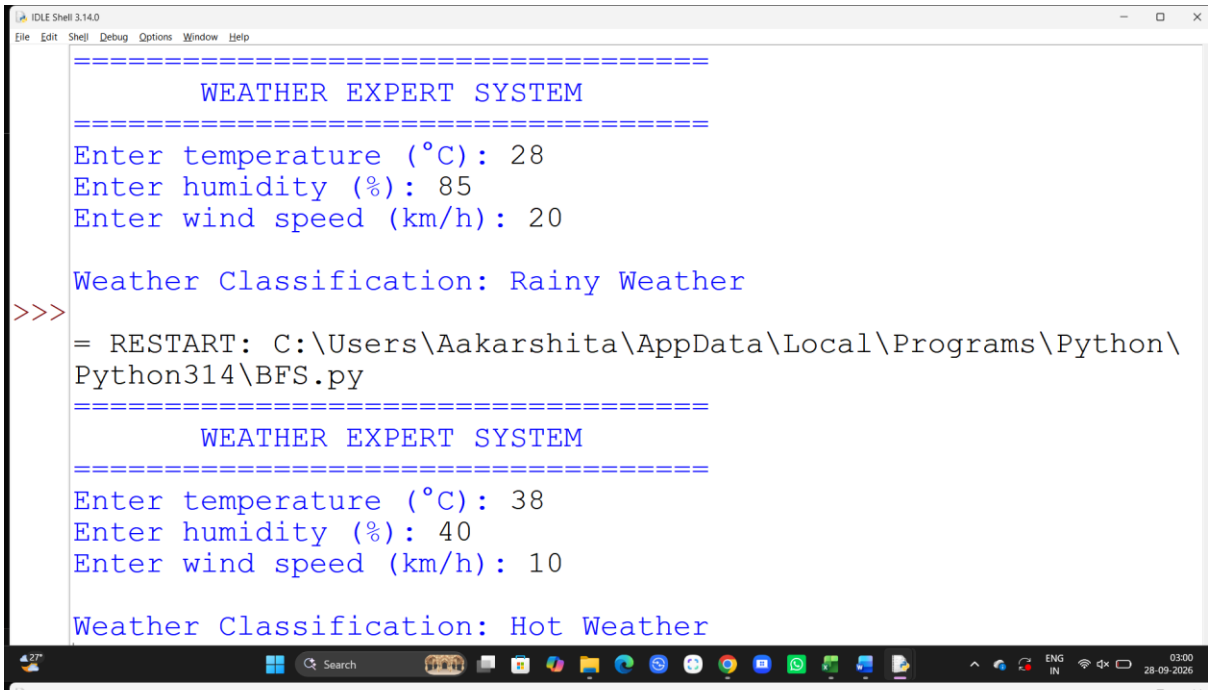
    else:
        print("Default rule fired: Moderate Weather")
        return "Moderate Weather"

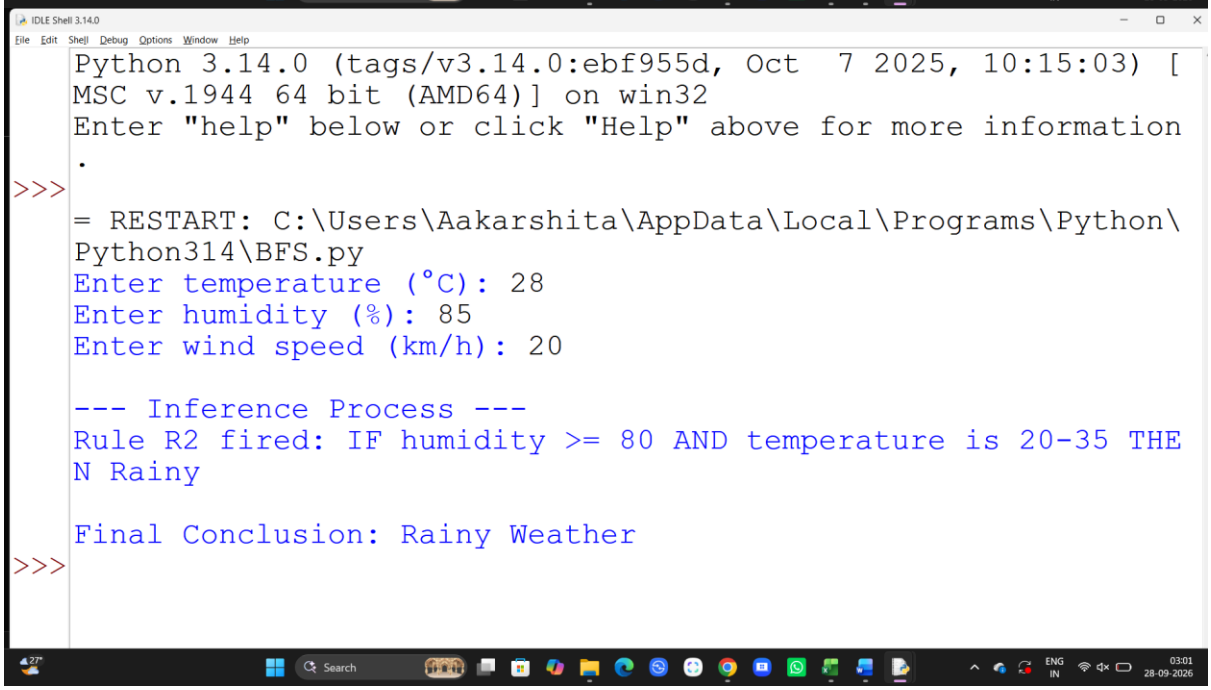

# User input
temp = float(input("Enter temperature (°C): "))
humidity = float(input("Enter humidity (%): "))
wind = float(input("Enter wind speed (km/h): "))

classification = weather_expert_system(temp, humidity, wind)

print("\nFinal Conclusion:", classification)
```

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
==========
WEATHER EXPERT SYSTEM
==========
Enter temperature (°C): 28
Enter humidity (%): 85
Enter wind speed (km/h): 20
Weather Classification: Rainy Weather
>>>
= RESTART: C:\Users\Aakarshita\AppData\Local\Programs\Python\Python314\BFS.py
==========
WEATHER EXPERT SYSTEM
==========
Enter temperature (°C): 38
Enter humidity (%): 40
Enter wind speed (km/h): 10
Weather Classification: Hot Weather


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Aakarshita\AppData\Local\Programs\Python\Python314\BFS.py
Enter temperature (°C): 28
Enter humidity (%): 85
Enter wind speed (km/h): 20
--- Inference Process ---
Rule R2 fired: IF humidity >= 80 AND temperature is 20-35 THEN Rainy
Final Conclusion: Rainy Weather
>>>

Practical No. 08

Aim:

To write a case study on Waymo's Self-Driving Car AI System.

Introduction:

Autonomous vehicles represent one of the most complex real-world applications of artificial intelligence, requiring systems to perceive, reason about, and act within unpredictable physical environments in real time. This case study examines Waymo, widely regarded as the leading company in commercial autonomous driving, to analyze the AI architecture, training methodology, and real-world performance of its self-driving system, known as the "Waymo Driver."

Background Information:

Waymo grew out of Google's self-driving project, which began in 2009 with founding members including Sebastian Thrun and Anthony Levandowski, and it became an independent Alphabet subsidiary in 2016. The company is currently led by co-CEOs Tekedra Mawakana and Dmitri Dolgov. Waymo operates a driverless ride-hailing service, Waymo One, across multiple U.S. cities, and by 2026 was actively expanding internationally.

System Architecture:

Sensor Fusion (Perception Layer):

The Waymo Driver combines cameras, lidar, and radar together with detailed pre-built maps to interpret its surroundings and make driving decisions. A journalist who test-rode a Waymo vehicle in Miami in early 2026 noted the car was equipped with 29 cameras and several sensors, including radar and lidar technology. This multi-sensor approach allows the system to cross-validate readings — lidar provides precise 3D distance data, radar performs well in poor weather and detects velocity, and cameras supply rich semantic detail (traffic lights, signage, lane markings) — which reduces the risk of any single sensor's blind spot compromising safety.

Decision-Making and Safety Verification:

A distinctive architectural feature is redundancy in the reasoning process itself. Waymo's engineering approach includes an independent onboard system that checks proposed trajectories against physical constraints and traffic laws before the vehicle acts on them — essentially a rules-based "safety checker" layered on top of the learned driving policy. ai

Waymo has also described a layered AI framework consisting of what it calls the "Driver," the "Simulator," and the "Critic" models working together, using a foundation-model approach it describes internally as a "Think Fast, Think Slow" system — combining fast, reactive driving responses with slower, more deliberate reasoning for complex scenarios.

Training Methodology:

Rather than relying solely on real-world miles, Waymo trains and validates its models heavily in simulation, using a "distillation" process and what it terms an "AI flywheel" — where data collected from real driving and simulated edge cases continuously retrains and refines the model. This approach lets the company test rare, high-risk scenarios (e.g., a child running into

the street) far more often than they would naturally occur on public roads. Waymo has also released the Waymo Open Dataset, giving outside researchers labeled sensor data, object trajectories, maps, and camera imagery to study perception and motion forecasting independently.

Real-World Deployment and Performance:

Waymo's scale has grown rapidly:

- The company reported 15 million rides in 2025, more than triple its prior annual volume, and as of August 2026 reported more than 500,000 fully autonomous trips per week. ai
- Waymo has completed more than 10 million fully driverless rides and cites an injury crash rate less than half that of human drivers.
- By 2026, Waymo operates in San Francisco, Los Angeles, Phoenix, Atlanta, and Austin, and was expanding to San Diego, Detroit, and Las Vegas.
- The company has also announced expansion into London, with public service planned for 2026, and began manual testing in Japan. Financially, Waymo raised $16 billion in 2026 at a $126 billion post-money valuation, with Alphabet remaining its majority investor — signalling strong market confidence in the technology's commercial viability.

Comparative Note: Waymo vs. Tesla's Approach:

It's worth noting the two dominant U.S. approaches to autonomous driving AI differ philosophically. Waymo uses a sensor-redundant approach (lidar + radar + camera + HD maps) with geofenced, fully driverless operation. Tesla's Full Self-Driving (FSD), by contrast, relies primarily on a camera-based, end-to-end neural network without lidar, aiming for a general-purpose system that works anywhere without pre-mapped routes, though still requiring driver supervision in most jurisdictions. This represents a classic AI engineering trade-off: Waymo prioritizes redundant, verifiable safety within a constrained domain, while Tesla bets on data scale and generalization across unconstrained environments.

Challenges and Limitations:

- **Edge cases**: Long-tail scenarios (unusual weather, construction zones, erratic human behaviour) remain the hardest problem for perception and prediction models.
- **Regulatory and public trust**: Expansion into new cities and countries depends heavily on local regulatory approval and public confidence, not just technical readiness.
- **Cost**: The sensor-heavy hardware stack (lidar, radar, numerous cameras) is significantly more expensive than camera-only approaches, a barrier to lower-cost scaling.
- **Localized incidents**: Even mature deployments experience friction — for instance, reports of Waymo vehicles clustering in specific neighborhoods due to routing quirks illustrate that even well-trained systems can produce unexpected emergent behavior at scale.

Conclusion:

Waymo's self-driving system illustrates how modern AI moves beyond single-model prediction into a full **system-of-systems**: sensor fusion, redundant safety-checking, simulation-driven training, and continuous real-world feedback loops ("flywheels") working together. Its rapid growth in ride volume and geographic reach through 2026 suggests that layered, safety-verified AI architectures — rather than any single breakthrough model — are what enable AI to operate reliably in high-stakes physical environments.